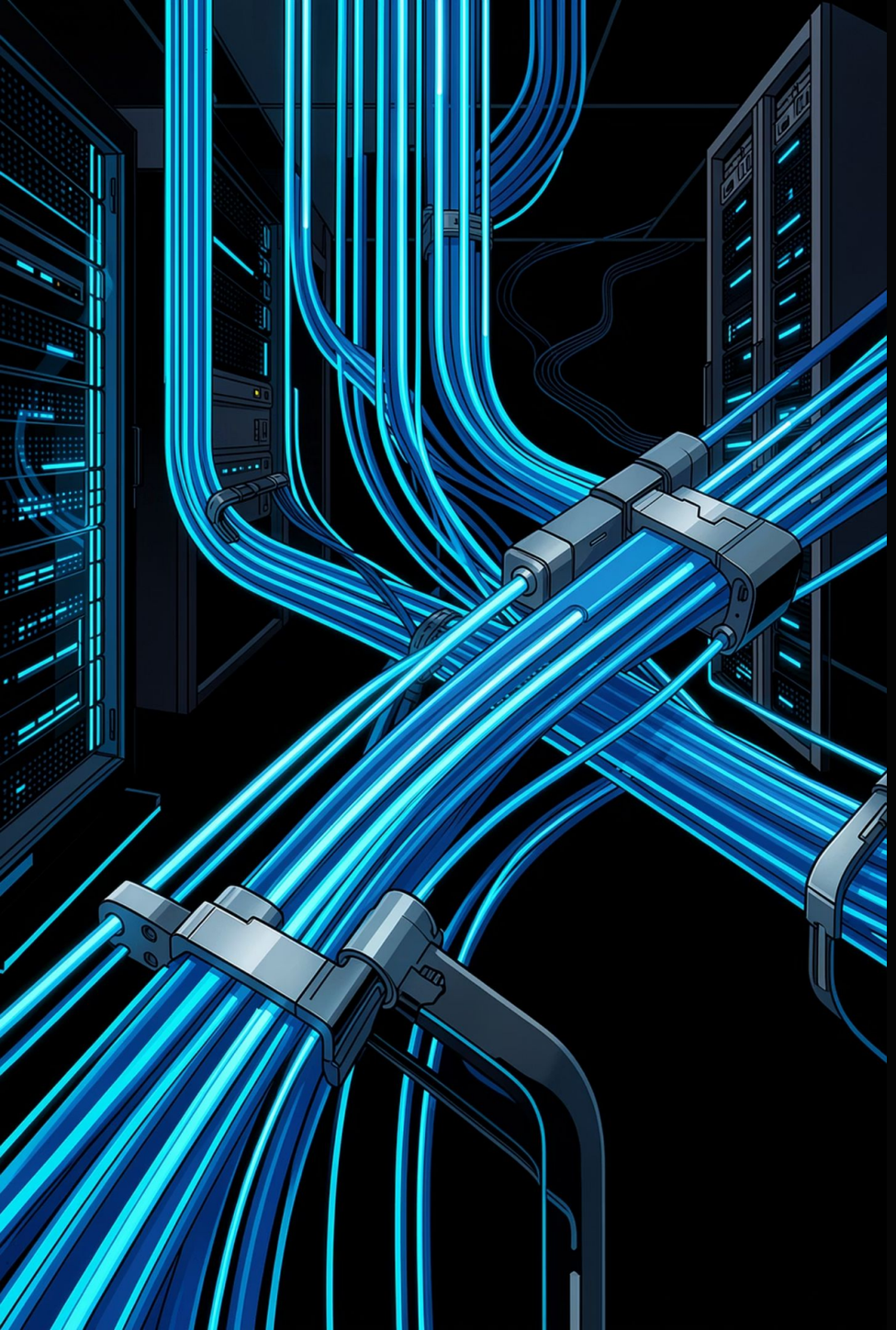


Reproducing BBR: A Controlled Study of Bottleneck-Based Congestion Control

CS 552 — Computer Networks · Spring 2026 · Rutgers University

Mihir Kulkarni · **Chaitanya Ranaware** · **Piyoosha Gadi** · Prof. Minsung Kim

We empirically reproduced Google's BBR algorithm against CUBIC and RENO on a controlled Linux testbed. Three experiments, one finding: BBR trades a small throughput gap for much lower latency.

The Problem: Loss-Based Congestion Control

For over 30 years, TCP congestion control has relied on packet loss as its main signal.

Buffer Bloat

A loss-based sender can fill router buffers before it sees a loss signal. Throughput stays high, but queuing delay rises and latency spikes.

The Core Question

Can we control congestion without using loss as the signal? BBR instead models bandwidth and propagation delay to target the optimal operating point.

BBR's Idea: Model the Path

BBR estimates bandwidth and propagation delay, then uses their product—the **Bandwidth-Delay Product (BDP)**—as the target for in-flight data.

BtlBw — Bottleneck Bandwidth

The highest delivery rate seen on the path. BBR paces to this rate to keep the link busy.

BDP = BtlBw × RTprop

The ideal amount of in-flight data. BBR aims for full link utilization with an empty queue.

RTprop — Propagation Delay

The minimum RTT observed with no queuing. It defines BBR's latency floor.

Four-Phase Cycle

STARTUP, DRAIN, PROBE_BW, and PROBE_RTT repeat to refresh estimates and adapt quickly.

Linux since 4.9; used in Google's WAN, YouTube, and QUIC.

What We Built: The Testbed

A reproducible Linux namespace testbed with tc shaping — no VMs, no virtualization noise.

Topology

Two namespaces run on one CachyOS host (kernel 6.19), linked by a **veth pair**.

ns1 — server at 10.0.0.1

ns2 — client at 10.0.0.2

tc tbf + netem emulate the bottleneck

TSO/GSO/GRO are disabled so queue limits match real packets.

Traffic Control Stack

tbf

Sets the bottleneck rate.

netem

Adds delay and queue limits.

Traffic uses `iperf3 -C <bbr|cubic|reno>` with JSON output every 100 ms.

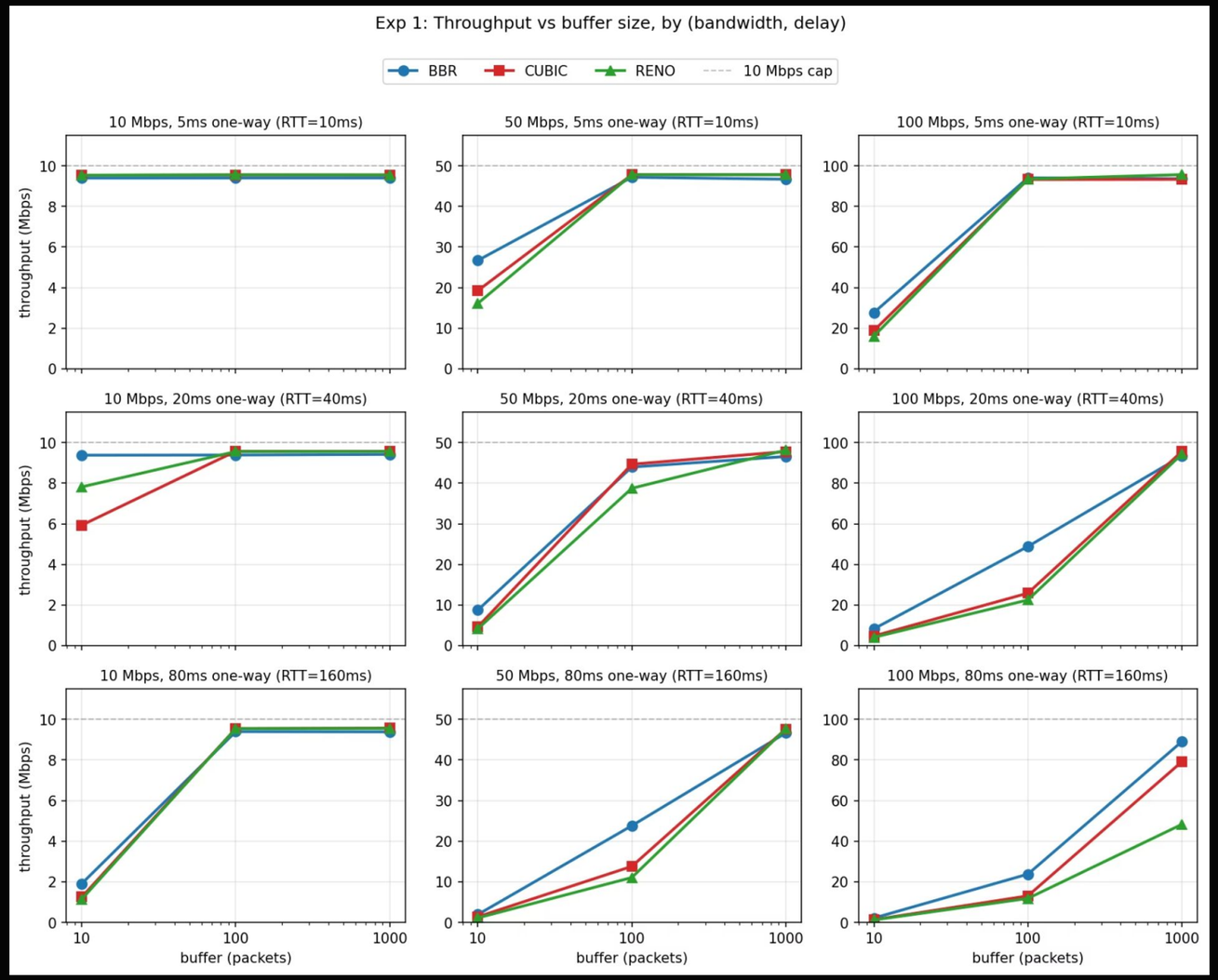
EXPERIMENT 1

Throughput Sweep: 81 Configurations

One-Way Delay / Buffer	BBR	CUBIC	RENO
5 ms (10 ms RTT) — any buffer	94%	93%	96%
20 ms (40 ms RTT) — 100 pkt	49%	26%	23%
80 ms (160 ms RTT) — 100 pkt	24%	13%	12%
80 ms (160 ms RTT) — 1000 pkt	89%	79%	48%

Key Finding: At shallow-to-moderate buffers on high-RTT links, all three CCs drop below capacity — but BBR consistently leads. On deep buffers, BBR matches CUBIC's throughput at a fraction of the latency cost (see Experiment 2). RENO never catches up at high RTT.

Exp 1: Throughput vs Buffer Size



EXPERIMENT 2

Buffer Bloat: Same Throughput, 3× the Latency

CC	Throughput	Mean RTT	RTT Inflation	Retransmits
BBR	92.9 Mbps	43.0 ms	1.07×	0
CUBIC	95.3 Mbps	120.3 ms	2.95×	93
RENO	95.3 Mbps	102.9 ms	2.53×	860

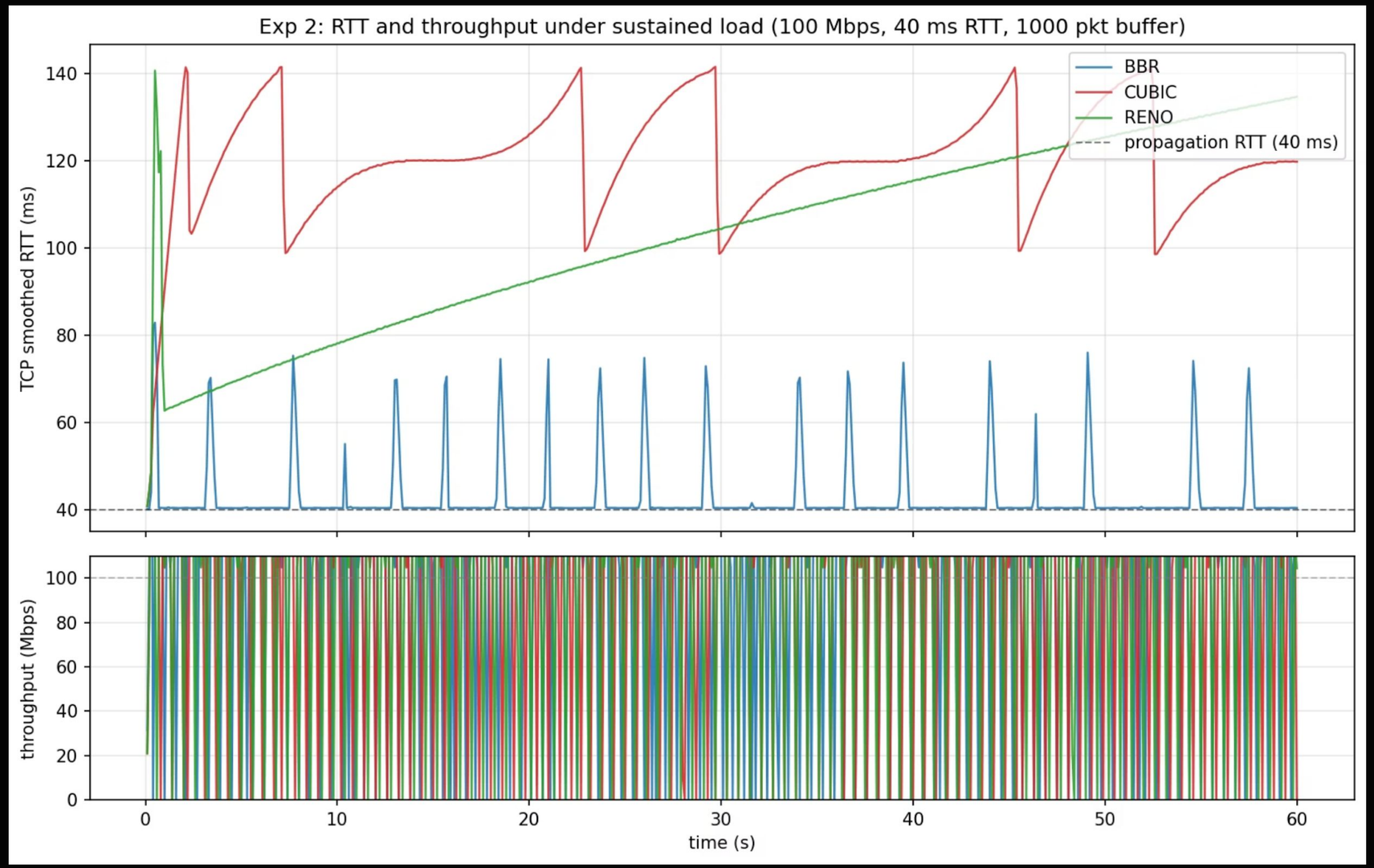
BBR's Advantage

BBR keeps latency at the propagation floor (~42 ms vs. 40 ms base). It never overfills the queue, resulting in **zero retransmits** in steady state.

Loss-Based Penalty

CUBIC and RENO inflate RTT by ~3× while achieving nearly identical throughput. RENO's 860 retransmits reflect aggressive queue overfilling and loss recovery cycles.

Exp 2: RTT and Throughput Under Sustained Load



EXPERIMENT 3

Fairness: Competing Flows on the Same Bottleneck

Pairing	Share Split	Jain's Index	Interpretation
BBR vs. CUBIC	0.55 / 0.45	0.990	Near-perfect fairness
BBR vs. BBR	0.52 / 0.48	0.999	Perfect intra-protocol fairness
CUBIC vs. CUBIC	0.53 / 0.47	0.997	Fair, as expected

BBR vs. BBR Baseline

Two BBR flows split bandwidth nearly perfectly (Jain = 0.999), the intra-protocol baseline.

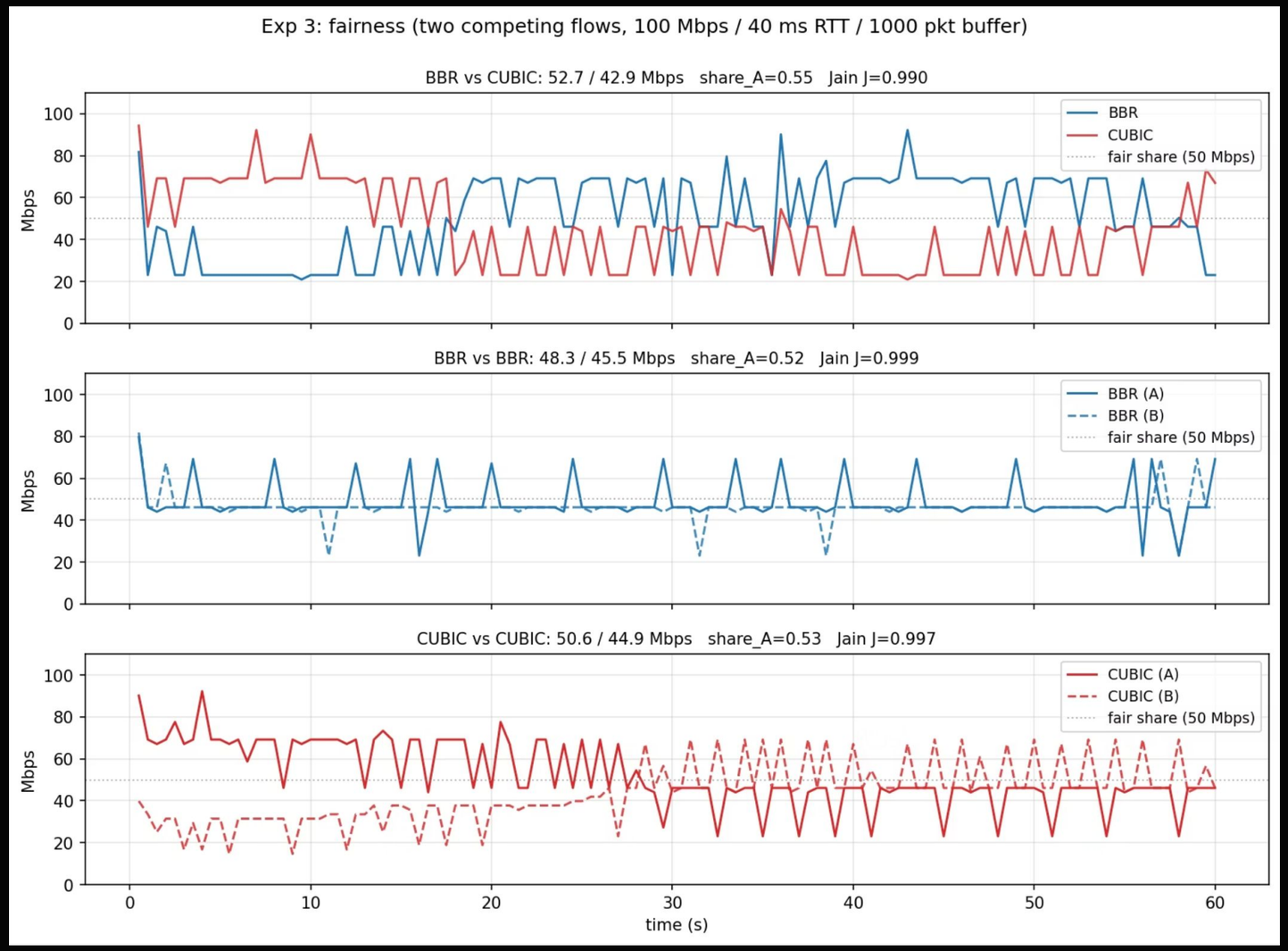
BBR vs. CUBIC: Fairer Than Expected

BBR took only a 55/45 share against CUBIC — far less unfair than the 2016 paper reported. Likely cause: kernel-side fairness patches accumulated in Linux BBR since publication.

BBRv2 Context

BBRv2 was developed to further tighten BBR-vs-CUBIC fairness in deep-buffer regimes, building on the original findings.

Exp 3: Fairness — Two Competing Flows



How Our Results Compare to the

✅ Reproduced Cleanly

- BBR beats CUBIC and RENO in all tested cases.
- Shallow buffers hurt CUBIC and RENO more.
- BBR keeps RTT near the propagation floor.
- BBR had no steady-state retransmits.

⚠️ Milder Than the Paper

- Linux fairness patches likely softened BBR's edge.
- Our 3×-BDP buffer is shallower than the paper's.
- BBR vs. CUBIC settled at 55/45 in our runs.
- Modern kernels reduce BBRv1 unfairness; BBRv2 tightens it further.

Note: We did not test lossy links or more than 2 flows, and we did not evaluate BBRv2.

Conclusions

1 **Loss-Based CC Is a Latency Tax**

BBR removes most of it by targeting a full link and empty queue, without waiting for loss.

2 **BBR's Win Is Context-Dependent**

It helps most on high-RTT, shallow-buffer paths and buffer-bloat-heavy links. On short-RTT paths with small buffers, the gap narrows.

3 **Fairness Holds in Modern Linux**

In our runs, BBR vs CUBC settled at 55/45 with Jain 0.990. The unfairness seen in the paper is now much less pronounced.

4 **A Laptop Testbed Suffices**

A controlled `netns` + `tc` setup reproduces the paper's qualitative claims. It is accessible, reproducible, and low-overhead.

> **Throughput isn't the whole story. Latency under load is what BBR fixes.**